

Slug Warrior



Practica 1

Programación de Juegos

Juan Jiménez Serrano

Luis Miguel Pérez Martín

Índice

1.Introducción	2
2. Primeras ideas	2
3. Escenas y contenido	4
3.1Primera escena	4
3.2 Segunda escena	5
3.3 Tercera escena	6
3.4 Cuarta escena	7
3.5 Quinta escena	8
3.6 Sexta escena	9
4. Técnicas de IA 's.	10
4.1 Máquina de estados.	10
4.2 Path tracking.	12
5. Scripts.	15
Dentro de la primera encontramos:	15
Dentro de la segunda encontramos:	15

1.Introducción

“Slug Warrior” es un juego 2D con estética medieval tipo Metal Slug pero con combate cuerpo a cuerpo.

Jugamos como una guerrera y tenemos que derrotar al jefe caballero que encontramos en la partida, disponemos de una espada y nuestra habilidad para derrotarlo antes de que nos mate.

2. Primeras ideas

Al principio los dos miembros del proyecto empezamos por separado, hasta que un día decidimos empezar el proyecto juntos, como los dos teníamos la misma idea de juego fue fácil decidirnos por qué tipo de juego hacer.

Teníamos en mente un juego tipo Metal Slug que la cámara te siga, tener distintas pantallas con diferentes enemigos y cuando derrotas a cada enemigo pasas a la siguiente pantalla.

Con el tiempo nos fuimos dando cuenta de que por una razón de tiempo el juego sería menos extenso de lo que nos gustaría, conseguimos implementar las mecánicas que queríamos pero en una versión menos densa con un escenario y un enemigo menos.

En cuanto al menú del juego y la interfaz tiene las cosas que en un principio pensamos y con más o menos tiempo conseguimos implementarlas y más adelante hablaremos de ellas.

3. Escenas y contenido

3.1 Primera escena



Es una pantalla muy sencilla, típica que encontramos en la mayoría de juegos, el título, algunas imágenes decorativas de la protagonista y un texto para poder pasar a la siguiente pantalla.

Tanto para esta pantalla como para el resto, el pasar de una escena a otra está implementado de la siguiente forma:

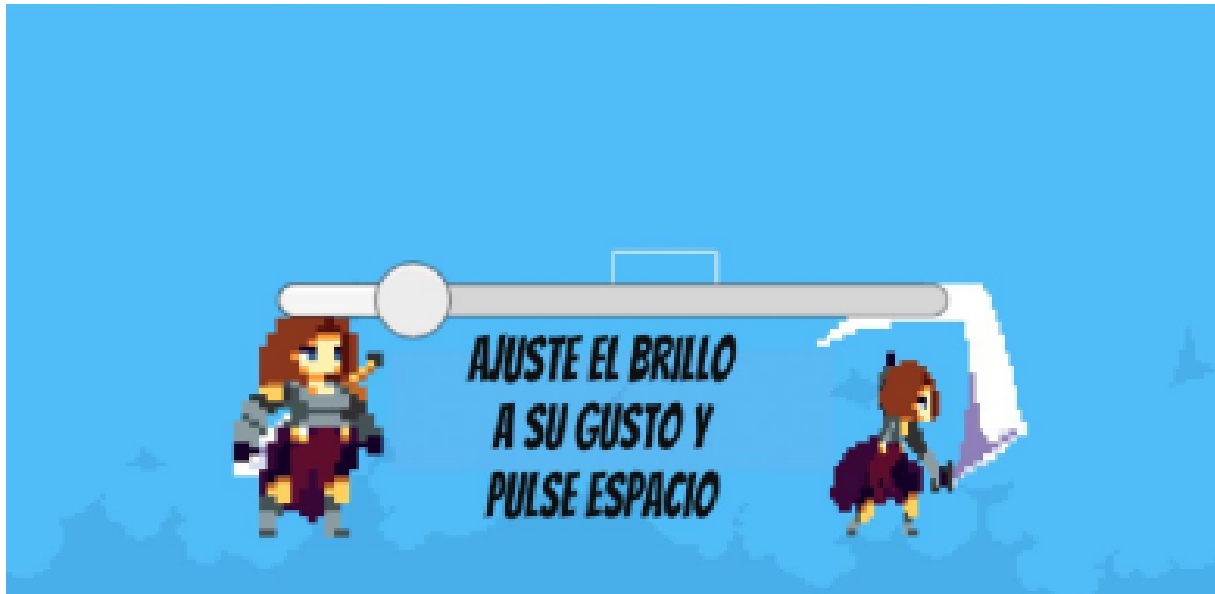
Utilizamos SceneManager y cargamos la escena deseada, para ellos tienen que estar añadidas en "Build Settings" de Unity.

```
SceneManager.LoadScene("BrilloSelect");
```

Para poder hacer esto necesitamos importar "SceneManagement".

```
using UnityEngine.SceneManagement;
```

3.2 Segunda escena



En la segunda pantalla encontramos el menú para ajustar el brillo del juego según se desee, aunque subirlo demasiado puede ser molesto.

En esta pantalla se crea el objeto "Brightness", tiene un componente Post-Process en el que podemos ajustar el valor "Auto-Exposure" para modificar el brillo.

A este objeto le hacemos un DontDestroyOnLoad para que se mantenga al cambiar de escenas. Esto se aplica también para la música de fondo que la creamos al principio.

3.3 Tercera escena



Este es el menú principal del juego, en el podemos iniciar partida, ir al menú de opciones, records y salir del juego.

Son 4 botones y cada uno de ellos te manda a una escena distinta excepto la opción de salir.

3.4 Cuarta escena

En esta escena, podemos realizar la configuración para nuestro juego.



Tenemos un ajustador de volumen y un selector de vidas entre 3, 5 y 7 vidas. Por otro lado tenemos 2 botones para Volver al Menú o ir al juego directamente.

El slider de Volumen funciona con un código que recoge el valor del slider y aplica ese valor al volumen del Audio Source que pasamos por public.

El selector de vidas utiliza el método `HandleInputData()` que obtiene el valor introducido en el Dropdown y lo guarda en las `PlayerPrefs()`.

3.5 Quinta escena



Nos situamos ya en el juego, tenemos un contador de tiempo, para visualizar el tiempo que llevamos en partida y posteriormente, guardaremos este tiempo en `PlayerPrefs`, para trabajar con ellos en la pantalla de récord. Esta escena comprueba si pulsamos la tecla escape para entrar en modo pausa.

Por otro lado vemos el sistema de vidas, en opciones elegimos el número de vidas entre 3, 5 y 7. Hemos puesto 7 sprites de corazones al lado del temporizador, mediante el script `playermovement`, utilizando el método `SetActive()`, hacemos que sean visibles una determinada cantidad de corazones en función de lo que hemos

escogido en las opciones. Posteriormente si recibimos daño o colisionamos con las calabazas que nos dan vida, desactivamos o activamos los corazones en función de nuestra vida actual.

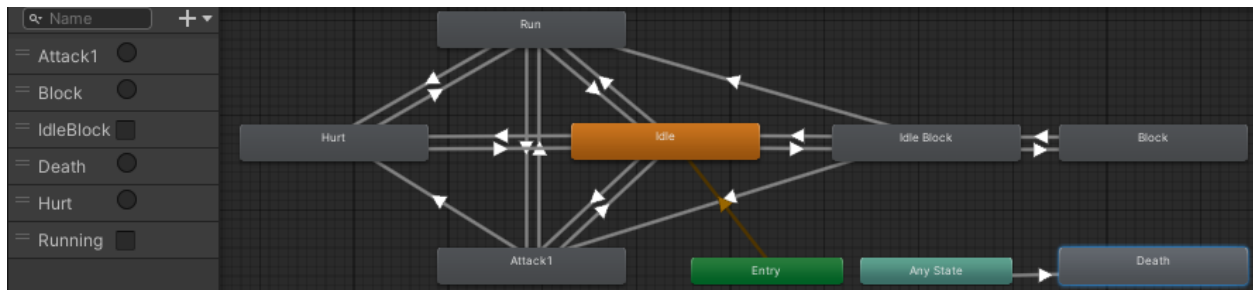
3.6 Sexta escena



Esta pantalla hace las comprobaciones necesarias con el tiempo que obtenemos al terminar la partida y calcula el puesto obtenido en función del tiempo y desde ella podremos volver al menú principal.

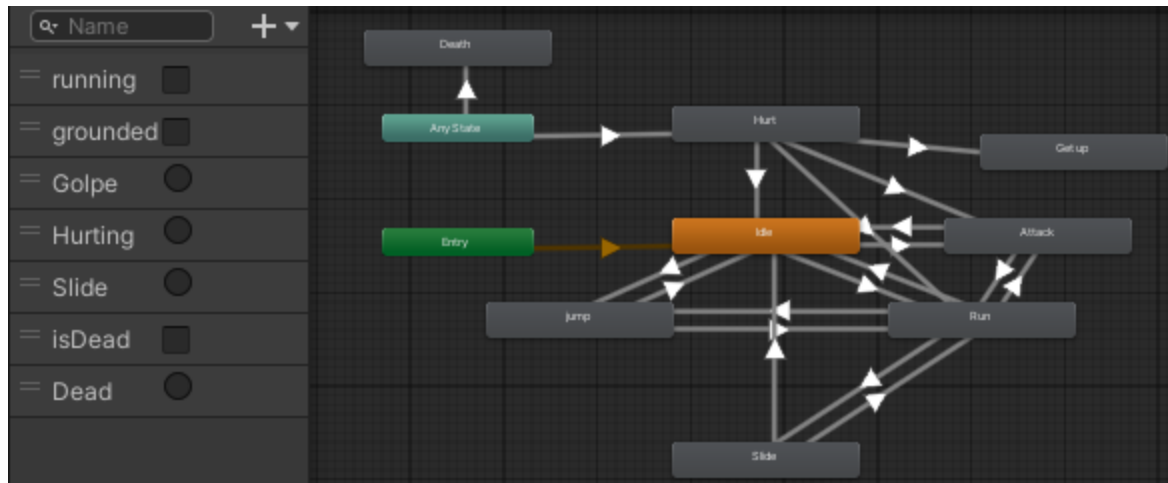
4. Técnicas de IA 's.

4.1 Máquina de estados.



Máquina de estado de nuestro enemigo, mediante el uso de Triggers y Booleanos hemos podido modelar el comportamiento de dicho enemigo. Obteniendo del jugador principal tanto su posición como cantidad de vida y el daño. en función de la distancia al jugador el enemigo esperara, avanzara, bloqueara o atacara, optando por un estilo pasivo de combate. Por otro lado cuando el jugador muere, este pasa de forma permanente en el idle, y si el jugador ataca a nuestro enemigo recibirá más o menos daño en función de si bloquea o no y el daño del jugador. Una vez la vida del enemigo es igual o inferior a 0, esté activa de forma permanente su estado de muerto.

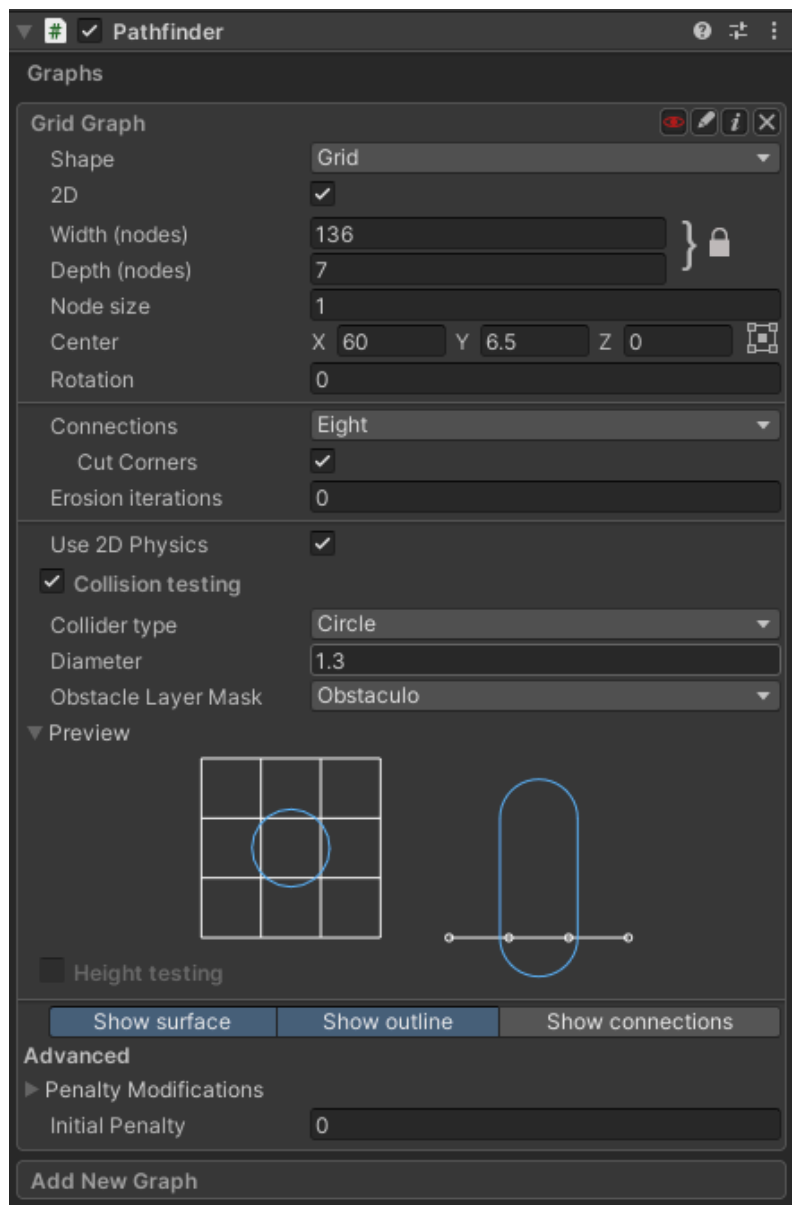
Haremos uso de un Animator, variables para gestionar la vida, dirección velocidad etc... y obtendremos el gameobject y el playermovement del jugador. Gracias a los cuales obtendremos su vida, daño y posición.



Máquina de estado de nuestro jugador, mediante el uso de Triggers y Booleanos hemos podido modelar el comportamiento de sus animaciones. Mediante las entradas recibidas por el jugador el personaje puede estar en el idle (ninguna), saltando, deslizándose, moviéndose o atacando, en caso de recibir daño se activará el estado Hurt y en función de si aun tiene vida o no, se levantara o pasará al estado permanente de vida.

Haremos uso del Rigidbody2D y el Animator, además de maxLife y la vida la cual mostraremos en pantalla por medio de distintos GameObject “Corazones” los cuales irán siguiendo la camara al estar “linkadas” para ejercer daño emplearemos el uso de Physics2D (concretamente usaremos el método `OverlapCircleAll` para comprobar si el enemigo está en el área).

4.2 Path tracking.



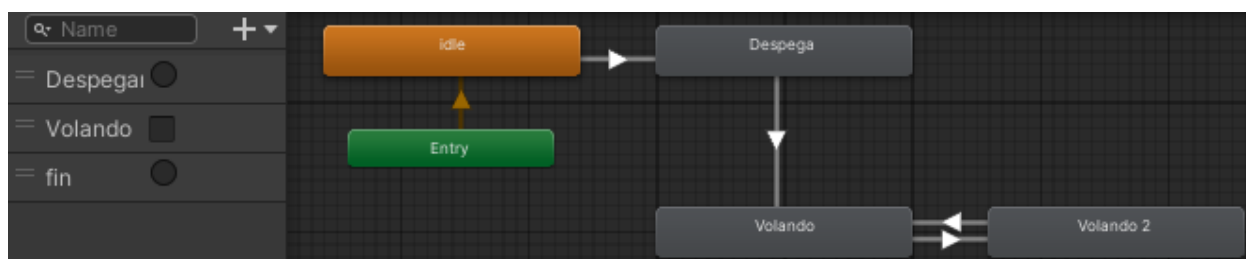
Mediante el uso del Asset de pathfinder y un objeto vacío al que insertarlo podemos crear un mapeado de las zonas que nuestra IA podrá o no ocupar mientras sigue a nuestro objetivo (Jugador).

Perseguirá a nuestro jugador, obteniendo su posición y actualizando constantemente, y al compararla con la de nuestro murciélago podrá ir generando en

función de dicho Pathfinder podrá ir optando por un camino u otro. (Pudiendo tener mapeado los obstáculos).



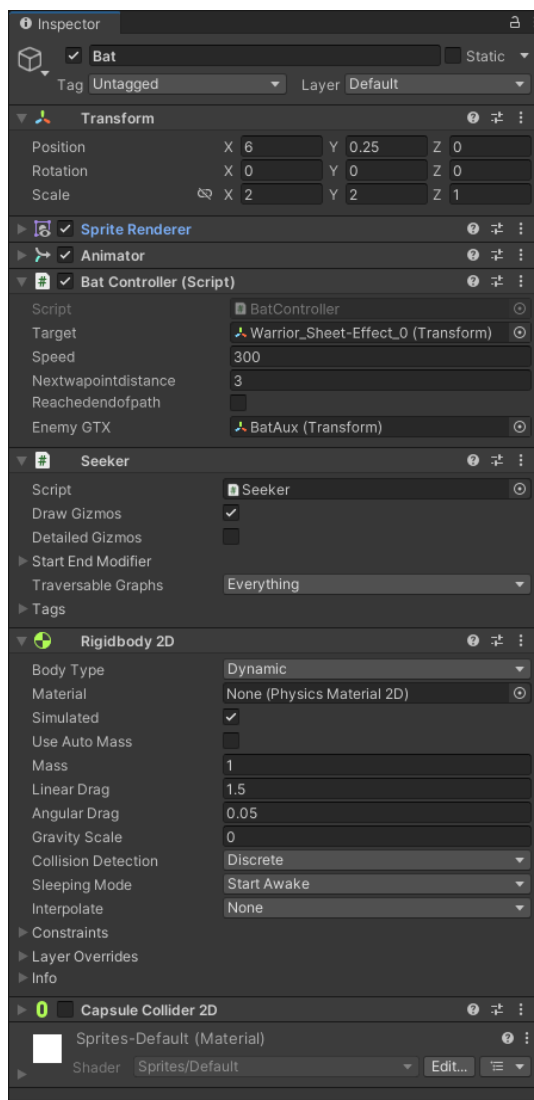
Como hemos mencionado anteriormente generamos un área de “Movimiento”, que está representada por la zona azul, en dicha área el murciélago se podrá mover libremente no pudiendo estar cerca del suelo ni atravesar las nubes.



Para terminar, generamos una máquina de estados muy simple en la cual el murciélago despegue desde su posición inicial, y se queda de forma permanente en los estados de vuelo que se encargaran de hacer un “loop” de vuelo.

Para el control del Murciélago hacemos uso de la librería.

using Pathfinding;



Al objeto murciélago le hemos incorporado tanto el script necesario para poder ir “Trackeando” la posición de nuestro personaje y el área de libre vuelo a través de los cuales poder generar los caminos que irá siguiendo nuestra IA para acompañar a nuestro personaje en su viaje.

5. Scripts.

Teniendo en cuenta que hay bastantes scripts comentaremos cada uno de ellos para facilitar la búsqueda.

Tenemos dos carpetas Scripts la que encontramos en la carpeta Assets y la que encontramos en la carpeta Scenes.

Dentro de la primera encontramos:

PlayerMovement: Contiene todo el código referente al personaje, vidas movimiento, daño, animaciones...

Enemigo: Código perteneciente al funcionamiento del enemigo.

CamaraScript: Código de la cámara, principalmente seguir al personaje.

Dentro de la segunda encontramos:

BrilloEscena:Script de pruebas del brillo.

BrilloManager:Código para el manejo del brillo:

ControlMenús:Script para pruebas.

Dropdown: Código para el seleccionador de vidas.

MainMenu: Código del menú principal.

OpcionesMenu: Código del menú de opciones.

PausaEscena: Código para la escena de pausa.

PausaInGame: Código para gestionar la pausa dentro del juego.

RecordsScript: Código para la escena de records.

SegundoSound: Código para el funcionamiento del slider de volumen.

SliderBrillo: Código para la escena de ajuste de brillo.

SoundManager: Script de prueba.

TextoInicio: Código para la pantalla de inicio.

TimerRecord: Código para el temporizador del juego.